

PLAYWRIGHT + POM + PORTFOLIO

Гайд по 20 автотестам для QA Automation портфолио

Пошаговая шпаргалка по твоему проекту Padel Community: куда класть тесты, как использовать fixtures и POM, какие ожидания писать и как должен выглядеть код.

Проект: padel-community

Стек: Playwright, TypeScript, Page Object Model, test fixtures

Главный принцип: тест описывает бизнес-сценарий, POM хранит детали UI.

Базовые правила

Reset уже централизован в `tests/fixtures/test.ts`. Если тест использует `page` или любой POM fixture, `reset` вызывается автоматически. Если тест использует только `request`, добавь `test.beforeEach(async ({ resetApp }) => await resetApp())`.

```
import { test, expect } from "../fixtures/test";
import { users, games, clubs, fullName } from "../data/test-data";

test.describe("Feature area", () => {
  test("[category] scenario name", async ({ loginPage, homePage }) => {
    await loginPage.loginAndExpectHome(users.sasha.email);
    await homePage.openGame(games.open);
  });
});
```

- Не импортируй `test` из `@playwright/test` в e2e specs - используй `./fixtures/test`.
- Не используй `waitForTimeout`. Жди URL, alert, visible/hidden, enabled/disabled, текст.
- Если локатор повторяется - добавь метод в POM.
- Если тест меняет БД - `reset` нужен. Сейчас он уже встроен в `page fixture`.
- Имена тестов оставляй бизнесовыми: что проверяем и какой риск закрываем.

Карта файлов

1. [functional] user can update profile and changes persist after relogin

tests/auth.spec.ts

2. [functional] user can leave waitlist from full match

tests/matches.spec.ts

3. [functional] organizer can edit game club and price is recalculated

tests/organizer.spec.ts

4. [functional] organizer can cancel an open game

tests/organizer.spec.ts

5. [functional] admin can change user role and new permissions are applied

tests/admin.spec.ts

6. [negative] non-admin cannot access admin users page

tests/admin.spec.ts

7. [negative] non-organizer cannot access organizer game management

tests/organizer.spec.ts

8. [negative] organizer cannot add the same player twice

tests/organizer.spec.ts

9. [negative] tournament results reject wins greater than matches

tests/admin.spec.ts

10. [negative] club creation validates required name city address and price

tests/admin.spec.ts

11. [regression] waitlisted player is promoted when participant leaves and order is preserved

tests/matches.spec.ts

12. [regression] completed tournament updates ranking after page refresh

tests/ranking.spec.ts

13. [regression] cancelled game deep link is read-only and not joinable

tests/matches.spec.ts

14. [api] POST /api/level-assessment saves valid assessment for authenticated user

tests/api.spec.ts

15. [api] POST /api/level-assessment rejects unauthenticated request

tests/api.spec.ts

16. [api] POST /api/test/reset rejects invalid reset token

tests/api.spec.ts

17. [smoke] guest can open home ranking login and register pages

tests/smoke.spec.ts

18. [responsive] mobile user can login open match details and navigate profile

tests/responsive.spec.ts

19. **[accessibility] critical pages have no blocking accessibility violations**

tests/accessibility.spec.ts

20. **[security] deactivated user cannot login**

tests/auth.spec.ts

20 тестов: подсказки и код

1. [functional] user can update profile and changes persist after relogin

tests/auth.spec.ts

Зачем: Проверяет end-to-end поток изменения профиля и сохранение данных после новой авторизации.

Подсказки:

- Используй ProfilePage, не пиши локаторы профиля прямо в тесте.
- После submit жди success alert или обновлённое значение в profile card.
- Persistence лучше проверять после logout/login, а не только сразу после save.

Полезный POM/helper:

```
// tests/pages/profile-page.ts
async expectProfileContains(...values: string[]) {
  for (const value of values) {
    await expect(this.card).toContainText(value);
  }
}
```

Пример теста:

```
test("[functional] user can update profile and changes persist after relogin", async ({
  loginPage,
  profilePage,
  page
}) => {
  await loginPage.loginAndExpectHome(users.sasha.email);

  await profilePage.goto();
  await profilePage.updateProfile({ city: "Tel Aviv", phone: "0501234567" });
  await expect(profilePage.success).toBeVisible();
  await profilePage.expectProfileContains("Tel Aviv", "0501234567");

  await page.getByTestId("avatar-menu-button").click();
  await page.getByTestId("logout-button").click();

  await loginPage.loginAndExpectHome(users.sasha.email);
  await profilePage.goto();
  await profilePage.expectProfileContains("Tel Aviv", "0501234567");
});
```

2. [functional] user can leave waitlist from full match

tests/matches.spec.ts

Зачем: Проверяет, что игрок может попасть в waitlist полного матча и затем выйти из него.

Подсказки:

- Используй full seeded game, чтобы не создавать матч вручную.
- Проверь badge On Waitlist перед leave.
- После leave проверь, что снова доступно действие Join Waitlist.

Полезный POM/helper:

```
// tests/pages/game-details-page.ts
async expectWaitlistActionAvailable() {
  await expect(this.joinButton).toHaveText("Join Waitlist");
  await expect(this.joinButton).toBeEnabled();
}
```

Пример теста:

```
test("[functional] user can leave waitlist from full match", async ({
  loginPage,
  homePage,
  gameDetailsPage
}) => {
  await loginPage.loginAndExpectHome(users.admin.email);
  await homePage.openGame(games.full);

  await gameDetailsPage.expectWaitlistActionAvailable();
  await gameDetailsPage.join();
  await gameDetailsPage.expectBadge("On Waitlist");

  await gameDetailsPage.leave();

  await gameDetailsPage.expectWaitlistActionAvailable();
});
```

3. [functional] organizer can edit game club and price is recalculated

tests/organizer.spec.ts

Зачем: Проверяет редактирование клуба/города и автопересчёт цены в форме игры.

Подсказки:

- Не кликай поля формы из теста - сделай метод updateGame в OrganizerPage.
- Жди redirect ?saved=game после сохранения.
- Проверь и address, и price, потому что оба зависят от выбранного клуба.

Полезный POM/helper:

```
// tests/pages/organizer-page.ts
async updateGame(input: { city?: string; clubLabel?: string; courtNumber?: string }) {
  if (input.city) await this.testId("game-city").selectOption(input.city);
  if (input.clubLabel) await this.testId("game-club").selectOption({ label: input.clubLabel });
  if (input.courtNumber) await this.testId("game-courtNumber").fill(input.courtNumber);

  await Promise.all([
    this.page.waitForURL(/\/organizer\/games\/[^\?]+\?saved=game$/),
    this.testId("game-save").click()
  ]);
  await expect(this.success).toHaveText("Game details saved.");
}
```

Пример теста:

```
test("[functional] organizer can edit game club and price is recalculated", async ({
  loginPage,
  organizerPage,
  page
}) => {
  await loginPage.loginAndExpectHome(users.organizer.email);
  await organizerPage.goto();
  await organizerPage.openGame(games.open);

  await organizerPage.updateGame({
    city: clubs.herzliya.city,
    clubLabel: clubs.herzliya.name
  });

  await expect(page.getByTestId("game-address")).toHaveValue(/Ramat Yam/i);
  await expect(page.getByTestId("game-price")).toHaveValue(/₪/);
});
```

4. [functional] organizer can cancel an open game

tests/organizer.spec.ts

Зачем: Проверяет базовый organizer flow изменения статуса игры на Cancelled.

Подсказки:

- У тебя dialogs автоматически accepted в fixture.
- После Cancel жди saved=status или badge Cancelled.
- Проверь, что Complete становится disabled.

Полезный POM/helper:

```
// tests/pages/organizer-page.ts
async cancelGame() {
  await Promise.all([
    this.page.waitForURL(/\/organizer\/games\/[^\?]+\?saved=status$/),
    this.gameAction("cancel").click()
  ]);
}
```

Пример теста:

```
test("[functional] organizer can cancel an open game", async ({ loginPage, organizerPage }) => {
  await loginPage.loginAndExpectHome(users.organizer.email);
  await organizerPage.goto();
  await organizerPage.openGame(games.open);

  await organizerPage.cancelGame();

  await expect(organizerPage.gameAction("complete")).toBeDisabled();
  await expect(organizerPage.gameAction("cancel")).toBeDisabled();
});
```

5. [functional] admin can change user role and new permissions are applied

tests/admin.spec.ts

Зачем: Проверяет RBAC: admin меняет роль игрока, и пользователь получает organizer permissions.

Подсказки:

- Используй AdminUsersPage.changeRole.
- После смены роли залогинься этим пользователем заново.
- Проверяй не только select, а реальное новое право доступа.

Полезный POM/helper:

```
// tests/pages/login-page.ts
async logout() {
  await this.page.getByTestId("avatar-menu-button").click();
  await this.page.getByTestId("logout-button").click();
  await expect(this.page).toHaveURL(/\/login\/$/);
}
```

Пример теста:

```
test("[functional] admin can change user role and new permissions are applied", async ({
  adminUsersPage,
  loginPage,
  organizerPage
}) => {
  await loginPage.loginAndExpectHome(users.admin.email);
  await adminUsersPage.goto();

  await adminUsersPage.changeRole(users.sasha.email, "ORGANIZER");
  await expect(adminUsersPage.success).toBeVisible();

  await loginPage.logout();
  await loginPage.loginAndExpectHome(users.sasha.email);
  await organizerPage.goto();

  await expect(organizerPage.newGameLink).toBeVisible();
});
```


6. [negative] non-admin cannot access admin users page

tests/admin.spec.ts

Зачем: Проверяет запрет доступа к admin users для обычного игрока.

Подсказки:

- Negative access tests лучше держать рядом с admin tests.
- Проверь итоговый URL или access denied message - зависит от реализации приложения.
- Не проверяй внутреннюю ошибку сервера; нужен пользовательский результат.

Пример теста:

```
test("[negative] non-admin cannot access admin users page", async ({ loginPage, page }) => {
  await loginPage.loginAndExpectHome(users.sasha.email);

  await page.goto("/admin/users");

  await expect(page).not.toHaveURL(/\/admin\/users$/);
  await expect(page.getByText(/not authorized|access denied|admin/i)).toBeVisible();
});
```

7. [negative] non-organizer cannot access organizer game management

tests/organizer.spec.ts

Зачем: Проверяет, что обычный игрок не может открыть organizer management.

Подсказки:

- Используй прямой URL, потому что nav link может быть скрыт.
- Для game id используй helper getGameIdByTitle.
- Проверь redirect или сообщение об отказе доступа.

Пример теста:

```
test("[negative] non-organizer cannot access organizer game management", async ({
  loginPage,
  page
}) => {
  await loginPage.loginAndExpectHome(users.sasha.email);

  const gameId = await getGameIdByTitle(games.open);
  await page.goto(`/organizer/games/${gameId}`);

  await expect(page).not.toHaveURL(new RegExp(`/organizer/games/${gameId}$`));
  await expect(page.getByText(/not authorized|access denied|organizer/i)).toBeVisible();
});
```

8. [negative] organizer cannot add the same player twice

tests/organizer.spec.ts

Зачем: Проверяет защиту от повторного добавления участника.

Подсказки:

- Если UI скрывает уже добавленного игрока, проверяй именно отсутствие option.
- Если есть API/action, можно тестировать ошибку через form/API.
- Тест должен отражать продуктовую логику: duplicate player невозможен.

Полезный POM/helper:

```
// tests/pages/organizer-page.ts
async expectAddPlayerOptionHidden(firstName: string) {
  await expect(this.testId(`admin-add-player-option-${firstName}`)).toHaveCount(0);
}
```

Пример теста:

```
test("[negative] organizer cannot add the same player twice", async ({ loginPage, organizerPage })
=> {
  await loginPage.loginAndExpectHome(users.admin.email);
  await organizerPage.goto();
  await organizerPage.openGame(games.open);

  await organizerPage.addPlayer(users.sasha.name);
  await organizerPage.expectParticipantText(fullName(users.sasha));

  await organizerPage.expectAddPlayerOptionHidden(users.sasha.name);
});
```

9. [negative] tournament results reject wins greater than matches

tests/admin.spec.ts

Зачем: Проверяет валидацию результатов турнира.

Подсказки:

- Создание турнира вынеси в helper/ПОМ, чтобы тест был коротким.
- Проверяй error alert, а не просто отсутствие success.
- Данные должны явно нарушать правило: wins > matches.

Пример теста:

```
test("[negative] tournament results reject wins greater than matches", async ({
  adminTournamentsPage,
  loginPage
}) => {
  await loginPage.loginAndExpectHome(users.admin.email);
  await adminTournamentsPage.goto();

  await adminTournamentsPage.createTournament({
    title: "Invalid Results Cup",
    startsAt: "2026-07-12T18:00",
    city: clubs.telAviv.city,
    club: clubs.telAviv.name
  });
  await adminTournamentsPage.addParticipants(users.michael.name, users.anastasia.name);

  await adminTournamentsPage.fillResult(users.michael.name, "1", "2");
  await adminTournamentsPage.saveResults();

  await expect(adminTournamentsPage.error).toContainText(/wins|matches/i);
});
```

10. [negative] club creation validates required name city address and price

tests/admin.spec.ts

Зачем: Проверяет required validation формы создания клуба.

Подсказки:

- Не обязательно проверять все browser validation messages, если они нативные.
- Для custom validation проверяй error alert.
- Если submit blocked HTML5 validation, проверяй что row не создан.

Пример теста:

```
test("[negative] club creation validates required name city address and price", async ({
  adminClubsPage,
  loginPage
}) => {
  await loginPage.loginAndExpectHome(users.admin.email);
  await adminClubsPage.goto();

  await adminClubsPage.createClub({
    name: "",
    city: "",
    address: "",
    price: ""
  });

  await expect(adminClubsPage.row("")).toHaveCount(0);
  await expect(adminClubsPage.error.or(adminClubsPage.createForm)).toBeVisible();
});
```

11. [regression] waitlisted player is promoted when participant leaves and order is preserved

tests/matches.spec.ts

Зачем: Защищает важную регрессию waitlist promotion.

Подсказки:

- Проверь состояние до действия и после действия.
- Если есть несколько waiting players, добавь проверку первого в списке.
- Не полагайся только на count.

Пример теста:

```
test("[regression] waitlisted player is promoted when participant leaves and order is preserved",
  async ({
    loginPage,
    homePage,
    gameDetailsPage
  }) => {
    await loginPage.loginAndExpectHome(users.boris.email);
    await homePage.openGame(games.full);

    await gameDetailsPage.expectWaitingPlayer(users.anton.name);

    await gameDetailsPage.leave();

    await gameDetailsPage.expectPlayer(users.anton.name);
    await gameDetailsPage.expectWaitingPlayerAbsent(users.anton.name);
  });
```

12. [regression] completed tournament updates ranking after page refresh

tests/ranking.spec.ts

Зачем: Проверяет, что ranking не только обновился, но и переживает refresh.

Подсказки:

- Можно переиспользовать поток создания турнира из admin tests.
- После completion переходи на ranking и делай page.reload().
- Проверь конкретного игрока и ожидаемые points.

Пример теста:

```
test("[regression] completed tournament updates ranking after page refresh", async ({
  adminTournamentsPage,
  loginPage,
  page
}) => {
  await loginPage.loginAndExpectHome(users.admin.email);
  await adminTournamentsPage.goto();

  await adminTournamentsPage.createTournament({
    title: "Ranking Refresh Cup",
    startsAt: "2026-07-12T18:00",
    city: clubs.telAviv.city,
    club: clubs.telAviv.name
  });
  await adminTournamentsPage.addParticipants(users.michael.name, users.anastasia.name);
  await adminTournamentsPage.quickFillMatches("1");
  await adminTournamentsPage.fillResult(users.michael.name, "1", "1");
  await adminTournamentsPage.fillResult(users.anastasia.name, "1", "0");
  await adminTournamentsPage.saveResults();
  await adminTournamentsPage.completeTournament();

  await page.goto("/ranking");
  await page.reload();

  await expect(page.getByTestId("ranking-player-Michael")).toContainText("27");
});
```

13. [regression] cancelled game deep link is read-only and not joinable

tests/matches.spec.ts

Зачем: Проверяет, что скрытая cancelled game всё равно защищена при прямом URL.

Подсказки:

- Это хороший пример deep-link regression.
- Используй gameIdByTitle.
- Проверь отсутствие join и leave actions.

Пример теста:

```
test("[regression] cancelled game deep link is read-only and not joinable", async ({
  loginPage,
  homePage,
  gameDetailsPage
}) => {
  await loginPage.loginAndExpectHome(users.diana.email);
  await homePage.expectGameHidden(games.cancelled);

  await gameDetailsPage.goto(await gameIdByTitle(games.cancelled));

  await gameDetailsPage.expectBadge("Cancelled");
  await gameDetailsPage.expectJoinUnavailable();
  await gameDetailsPage.expectLeaveUnavailable();
});
```

14. [api] POST /api/level-assessment saves valid assessment for authenticated user

tests/api.spec.ts

Зачем: Показывает, что ты умеешь тестировать API в authenticated context.

Подсказки:

- Для authenticated API используй page.request после UI login.
- Если endpoint меняет состояние, reset нужен перед тестом.
- Проверь и response.ok, и последующее состояние UI/API.

Пример теста:

```
test("[api] POST /api/level-assessment saves valid assessment for authenticated user", async ({
  loginPage,
  page,
  profilePage
}) => {
  await loginPage.loginAndExpectHome(users.sasha.email);

  const response = await page.request.post("/api/level-assessment", {
    data: { score: 14, level: 3.5 }
  });

  expect(response.ok()).toBeTruthy();

  await profilePage.goto();
  await expect(profilePage.card).toContainText("level 3.5");
});
```

15. [api] POST /api/level-assessment rejects unauthenticated request

tests/api.spec.ts

Зачем: Проверяет security boundary API без cookies/session.

Подсказки:

- Этот тест использует request, значит reset вызывай явно в beforeEach.
- Используй expectJsonError helper.
- Не логинься в этом тесте.

Пример теста:

```
import { test, expect } from "../fixtures/test";
import { expectJsonError } from "../utils/api";

test.beforeEach(async ({ resetApp }) => {
  await resetApp();
});

test("[api] POST /api/level-assessment rejects unauthenticated request", async ({ request }) => {
  const response = await request.post("/api/level-assessment", {
    data: { score: 14, level: 3.5 }
  });

  await expectJsonError(response, 401);
});
```

16. [api] POST /api/test/reset rejects invalid reset token

tests/api.spec.ts

Зачем: Проверяет, что test-only reset endpoint защищён токеном.

Подсказки:

- Не вызывай resetApp beforeEach именно для этого теста, если хочешь проверить invalid token первым действием.
- Ожидаемый статус обычно 401 или 403.
- Не логируй реальные секреты.

Пример теста:

```
test("[api] POST /api/test/reset rejects invalid reset token", async ({ request }) => {
  const response = await request.post("/api/test/reset", {
    headers: { "x-test-token": "wrong-token" }
  });

  expect(response.status()).toBe(401);
});
```


17. [smoke] guest can open home ranking login and register pages

tests/smoke.spec.ts

Зачем: Быстрый smoke для критичных public pages.

Подсказки:

- Smoke должен быть коротким и стабильным.
- Проверь ключевой элемент страницы, не весь контент.
- Этот тест можно запускать отдельно перед полным regression.

Пример теста:

```
test("[smoke] guest can open home ranking login and register pages", async ({ page }) => {
  await page.goto("/");
  await expect(page.getByTestId("games-list")).toBeVisible();

  await page.goto("/ranking");
  await expect(page.getByTestId("ranking-search")).toBeVisible();

  await page.goto("/login");
  await expect(page.getByTestId("login-form")).toBeVisible();

  await page.goto("/register");
  await expect(page.getByTestId("register-form")).toBeVisible();
});
```

18. [responsive] mobile user can login open match details and navigate profile

tests/responsive.spec.ts

Зачем: Проверяет основной пользовательский поток на мобильном viewport.

Подсказки:

- Используй test.use только в этом файле или describe block.
- Не проверяй pixel-perfect layout, проверяй доступность основных действий.
- Если nav отличается на mobile, добавь методы в POM.

Пример теста:

```
test.use({ viewport: { width: 390, height: 844 } });

test("[responsive] mobile user can login open match details and navigate profile", async ({
  loginPage,
  homePage,
  page
}) => {
  await loginPage.loginAndExpectHome(users.sasha.email);

  await homePage.openGame(games.open);
  await expect(page.getByRole("heading", { name: games.open })).toBeVisible();

  await page.getByTestId("avatar-menu-button").click();
  await page.getByTestId("nav-profile").click();

  await expect(page.getByTestId("profile-card")).toBeVisible();
});
```

19. [accessibility] critical pages have no blocking accessibility violations

tests/accessibility.spec.ts

Зачем: Показывает зрелость QA-подхода: базовая accessibility проверка критичных страниц.

Подсказки:

- Нужно установить @axe-core/playwright.
- Начни с нескольких critical pages, не сканируй весь сайт.
- В портфолио укажи, что violations triage делается отдельно.

Пример теста:

```
import AxeBuilder from "@axe-core/playwright";
import { test, expect } from "../fixtures/test";

for (const path of ["/", "/ranking", "/login", "/register"]) {
  test(`[accessibility] ${path} has no blocking accessibility violations`, async ({ page }) => {
    await page.goto(path);

    const results = await new AxeBuilder({ page })
      .withTags(["wcag2a", "wcag2aa"])
      .analyze();

    expect(results.violations).toEqual([]);
  });
}
```

20. [security] deactivated user cannot login

tests/auth.spec.ts

Зачем: Проверяет, что отключённый пользователь не может получить session.

Подсказки:

- Нужен seeded deactivated user или test helper/API для деактивации.
- Проверяй ошибку login и отсутствие перехода на home.
- Это security test, поэтому сценарий должен быть очень явным.

Пример теста:

```
test("[security] deactivated user cannot login", async ({ loginPage, page }) => {
  await loginPage.loginAs("deactivated@padel.test");

  await loginPage.expectInvalidCredentials();
  await expect(page.getByTestId("games-list")).toHaveCount(0);
});
```

Чеклист перед commit

- `npm run typecheck` проходит.
- `npx playwright test tests/имя.сpec.ts` проходит для нового файла.
- `npm run test:e2e` проходит перед финальной отправкой.
- Тест не зависит от результата предыдущего теста.
- Новые действия вынесены в POM, если они повторяются.
- Тестовые данные взяты из `tests/data/test-data.ts`, а не разбросаны строками.
- В API-only тестах `reset` вызван явно.